

В лабораторных работах 6-9 не нужно выигрывать время, как в предыдущих. Требуется «смоделировать описанную ситуацию», т.е. найти и реализовать алгоритм, успешно справляющийся с ситуацией (без тупиков и «необслуженных» (проигнорированных) процессов). Работы 6-9 выполните с помощью одного из инструментов (OpenMP или MPI).

Лабораторная работа №6 «Производители–Потребители»

В научно-технической литературе существует достаточно большое количество вариантов постановки данной задачи. В наиболее простом случае предполагается, что существует два потока, один из которых (*производитель*) генерирует сообщения (*изделия*), а второй поток (*потребитель*) их принимает для последующей обработки. Поток взаимодействуют через некоторую область памяти (*хранилище ограниченного размера*), в которой производитель размещает свои генерируемые сообщения и из которой эти сообщения извлекаются потребителем (см. рис. 1).

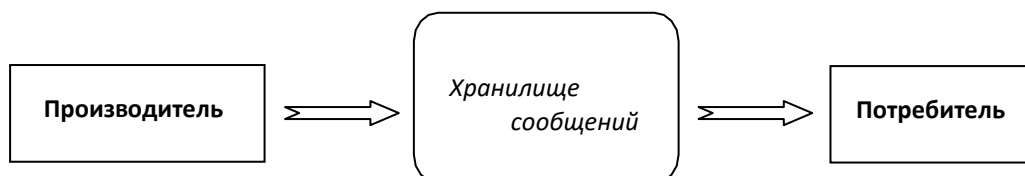


Рис. 1. Общая схема задачи «Производители–Потребители»

Рассмотрев постановку данной задачи, можно заметить, что хранилище сообщений представляет собой не что иное, как общий разделяемый ресурс, и использование этого ресурса должно быть построено по правилам взаимного исключения. Кроме того, следует учитывать, что потребление ресурса иногда может оказаться невозможным (отсутствие сообщений в хранилище), а при добавлении сообщений в хранилище могут происходить задержки (в случае полного заполнения хранилища).

Выходом программы является протокол в консоли из сообщений процессов участников, по которому можно оценить (понять) успешное/неуспешное решение задачи. Это относится ко всем лаб. работам с номерами с 6 по 9.

Лабораторная работа №7 «Читатели–Писатели»

Возможная простая постановка этой задачи состоит в следующем. Пусть имеется некоторая область памяти – хранилище данных, с которым одновременно работают несколько потоков. По характеру использования хранилища потоки могут быть разделены на два типа. Одна группа – это *потоки-читатели*, которые осуществляют только чтение (без удаления) хранилища. Другая – оставшаяся часть потоков – это *потоки-писатели*, которые выполняют запись новых значений в хранилище данных (см. рис. 2).

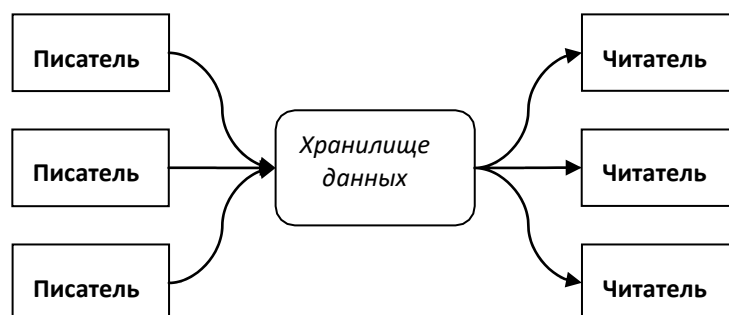


Рис. 2. Общая схема задачи «Читатели–Писатели»

При рассмотрении этой задачи предполагается, что потоки-читатели не изменяют каких-либо параметров хранилища и могут, тем самым, работать одновременно, не мешая друг другу. Для потоков-писателей ситуация обратная – предполагается, что запись не может выполняться несколькими потоками одновременно и, понятно, во время записи не допускаются какие-либо операции чтения.

Постановка задачи может различаться в правилах разрешения ситуации обращения потока-писателя к хранилищу. Если при попытке записи имеются активные потоки-читатели, то поток-писатель должен быть заблокирован до завершения работы потоков-читателей. Вопрос состоит в том, что делать с новыми поступающими запросами на чтение. Возможный вариант – отдать предпочтение потокам-читателям (т. е. новые потоки-читатели могут начинать свою работу, не обращая внимания на заблокированный процесс-писатель), однако в этом случае блокировка потока-писателя может продолжаться бесконечно долго. Альтернативный подход – блокировка новых потоков-читателей, появившихся после блокировки потока-писателя.

Хранилище не ограничено, каждый поток-читатель должен прочитать каждое сообщение, оставленное потоком-писателем.

Лабораторная работа №8 «Обедающие философы»

Данная задача является одной из наиболее известных в области параллельного программирования. Если задача «Читатели–Писатели» помогает демонстрировать методы параллельного и исключительного доступа к одному общему ресурсу, то задача «Обедающие философы» позволяет рассмотреть способы доступа нескольких потоков к нескольким разделяемым ресурсам.

Исходная формулировка задачи, впервые предложенная Э. Дейкстрой, выглядит следующим образом. Представляется ситуация, в которой пять философов располагаются за круглым столом. При этом философы либо размышляют, либо кушают. Для приема пищи в центре стола большое блюдо с неограниченным количеством спагетти, и тарелки, по одной перед каждым философом. Предполагается, что поесть спагетти можно только с использованием двух вилок. Для этого на столе располагается ровно пять вилок – по одной между тарелками философов (см. рис. 3).

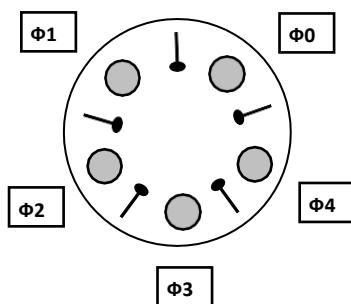


Рис. 3. Общая схема задачи «Обедающие философы»

Для того, чтобы приступить к еде, философ должен взять вилки слева и справа (если они не заняты), положить спагетти из большого блюда в свою тарелку, поесть, а затем обязательно положить вилки на свои места для их повторного использования (проблема чистоты вилок в задаче не рассматривается).

Нетрудно заметить, что в данной задаче философы представляют собой потоки, а вилки – общие разделяемые ресурсы. Тогда первое очевидное, на первый взгляд, решение состоит в том, чтобы для каждой вилки (ресурса) ввести отдельный семафор для блокировки философа (потока) в ситуации, когда нужная для еды вилка уже занята соседним философом. Кроме того, можно применить некоторое регламентирующее правило порядка взятия вилок – например, философ сначала берет левую вилку, затем правую.

Лабораторная работа №9 «Спящий парикмахер»

Данная задача также в числе широко используемых примеров для демонстрации проблем синхронизации. На примере этой задачи можно показать методы последовательного доступа к набору разделяемых ресурсов и рассмотреть организацию вычислений в соответствии со схемой «клиент– сервер».

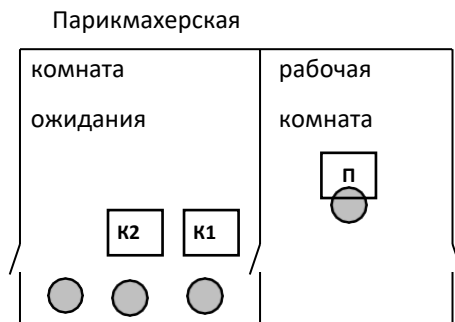


Рис. 4. Общая схема задачи «Спящий парикмахер» (К – клиенты, П – парикмахер)

Смысловая окраска задачи «Спящий парикмахер» состоит в следующем (см. рис. 4). Обсуждается проблема обслуживания клиентов парикмахерской. В парикмахерской имеется два помещения: комната ожидания, в которой ограниченное количество мест, и рабочая комната с единственным креслом, в котором располагается обслуживаемый клиент. Посетители заходят в парикмахерскую. Если комната ожидания заполнена, то поворачиваются и уходят; иначе занимают свободные места и засыпают, ожидая своей очереди к парикмахеру. Парикмахер, если есть клиенты, приглашает одного из них в рабочую комнату (при этом освобождается место в комнате ожидания) и подстригает его. После стрижки клиент покидает парикмахерскую, а парикмахер приглашает следующего посетителя и т. д. Если клиентов нет (комната ожидания пуста), парикмахер садится в свое рабочее кресло и засыпает. Будит его очередной появляющийся посетитель парикмахерской.

В данной задаче ресурсами являются места ожидания и рабочее кресло. Поток клиентов должны получать эти ресурсы строго последовательно: сначала посетитель должен найти место в комнате ожидания и только затем занять очередь к парикмахеру. При этом предоставление рабочего кресла для обслуживания производит специальный процесс-парикмахер. В этом плане, парикмахера можно интерпретировать как сервер, предоставляющий требуемый сервис.